

Dataintegration och verktyg

Magnus Ahlström

Systemmentor

Agenda för idag

- **Vad är dataintegration?** – Grundläggande definitioner.
- **Integrationsmetoder** – ETL, API, Events och Streaming.
- **Realtid vs Batch** – När ska man använda vad?
- **OpenAPI & Dokumentation** – Att skriva tydliga kontrakt.
- **Mocking & Simulering** – Utveckling utan fysisk hårdvara.
- **Verktyg** – n8n, Postman Flows och arkitekturer.

Vad är dataintegration?

Dataintegration handlar om att kombinera data från olika källor för att ge användaren en enhetlig vy eller för att låta system samarbeta.

- **Datakällor:** Sensorer, databaser, externa API:er, loggfiler.
- **Mål:** Att skapa värde genom att sammanställa information.
- **Utmaning:** Olika format, olika hastigheter, olika protokoll.

Varför behöver vi integrera?

1. **Bryta ner datasilon:** Data ska inte sitta fast i ett enskilt system.
2. **Automatisering:** Låta system trigga varandra (t.ex. "Om temp > 25, starta fläkt").
3. **Bättre beslutsunderlag:** Se trender genom att kombinera historisk data med realtidsdata.
4. **Skalbarhet:** Kunna lägga till nya sensorer eller tjänster utan att bygga om allt.

Integrationsmetoder: ETL (Batch)

ETL står för **Extract, Transform, Load**.

- **Extract:** Hämta data från källsystemet.
- **Transform:** Tvätta, formatera och räkna om data.
- **Load:** Skriva in data i målsystemet (t.ex. ett Data Warehouse).
- **Batch:** Körs ofta på schema (t.ex. en gång per natt).

API-baserad integration

Request/Response – Systemen pratar direkt med varandra.

- **Synkront:** Klienten frågar, servern svarar direkt.
- **REST & JSON:** Det vanligaste sättet i moderna webbsystem.
- **Användning:** När man behöver specifik data "här och nu".

Events & Streaming (Real-time)

Event-driven Architecture – Något händer, och informationen skickas direkt.

- **Asynkront:** Avsändaren skickar och går vidare, mottagaren läser när den kan.
- **Pub/Sub:** En producent "publicerar", många konsumenter kan "prenumerera".
- **Streaming:** Ett konstant flöde av händelser (t.ex. Kafka, MQTT).

Batch vs Realtid

Egenskap	Batch (ETL)	Realtid (Streaming/API)
Latens	Minuter till timmar	Millisekunder till sekunder
Datavolym	Stora mängder på en gång	Kontinuerligt flöde
Komplexitet	Ofta enklare att hantera	Kräver mer infrastruktur
Exempel	Månadsrapport	Inbrottslarm

API-dokumentation & OpenAPI

För att integrationen ska fungera krävs ett **kontrakt**.

- **OpenAPI (f.d. Swagger):** En standard för att beskriva REST API:er.
- **Maskinläsbart:** Skrivs i YAML eller JSON.
- **Dokumentation:** Kan generera snygga webbsidor där man kan testa anrop.

Varför använda OpenAPI?

1. **Single Source of Truth:** Dokumentationen matchar alltid verkligheten.
2. **Generering:** Skapa kodmallar för klienter och servrar automatiskt.
3. **Testning:** Validera att data som skickas faktiskt följer reglerna.
4. **Samarbete:** Utvecklare kan börja bygga mot specifikationen innan API:et ens är klart.

Mocking & Simulering

I IoT-utveckling har vi inte alltid hårdvaran tillgänglig.

- **Mocking:** Att skapa en "fejkad" version av en komponent (t.ex. en sensor).
- **Syfte:** Testa mjukvaran utan att behöva fysiska sensorer.
- **Kontrakt:** Använd OpenAPI för att se till att din mock skickar *exakt* rätt format.

Verktyg för integration

Man behöver inte alltid skriva all kod själv.

- **n8n / Node-RED:** Visuella verktyg för att koppla ihop olika tjänster (Low-code).
- **Postman Flows:** Visualisera API-kedjor.
- **Message Brokers:** MQTT (Mosquitto), RabbitMQ, Kafka.

Arkitektur: Från sensor till moln

Arkitektur

1. **Sensor:** Samlar in och skickar (Push).
2. **API:** Validerar (via OpenAPI) och tar emot.
3. **Database:** Lagrar historik persistent.
4. **Viewer:** Hämtar och visualiserar (Pull).

Sammanfattning

- **Dataintegration** är limmet mellan system.
- **ETL** för stora mängder historisk data (Batch).
- **API/Streaming** för snabbhet och händelser (Realtid).
- **OpenAPI** är standarden för att dokumentera och validera våra kontrakt.
- **Mocking** låter oss utveckla snabbare och säkrare utan hårdvara.

Dags för laboration!

Nu ska vi omsätta teorin i praktiken:

1. Definiera ett API-kontrakt med OpenAPI.
2. Uppdatera vårt API för att spara data i en SQLite-databas.
3. Bygga en mock-sensor som följer kontraktet.
4. Visualisera insamlad data.